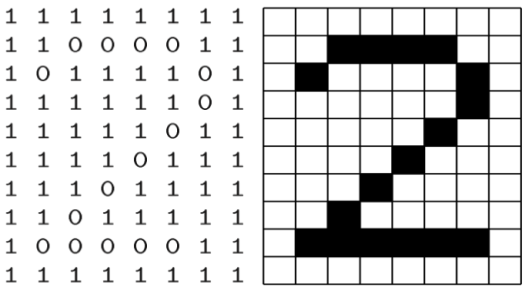
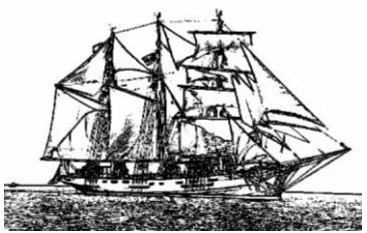
MAP201 : Découverte des Mathématiques Appliquées

Images

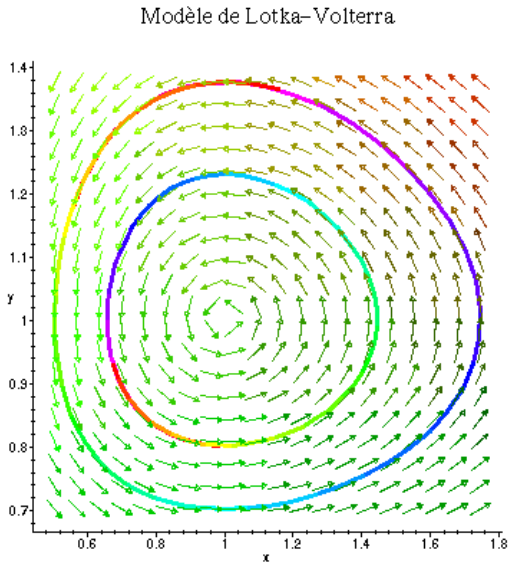






Jérome Lesaint

Equa Diff

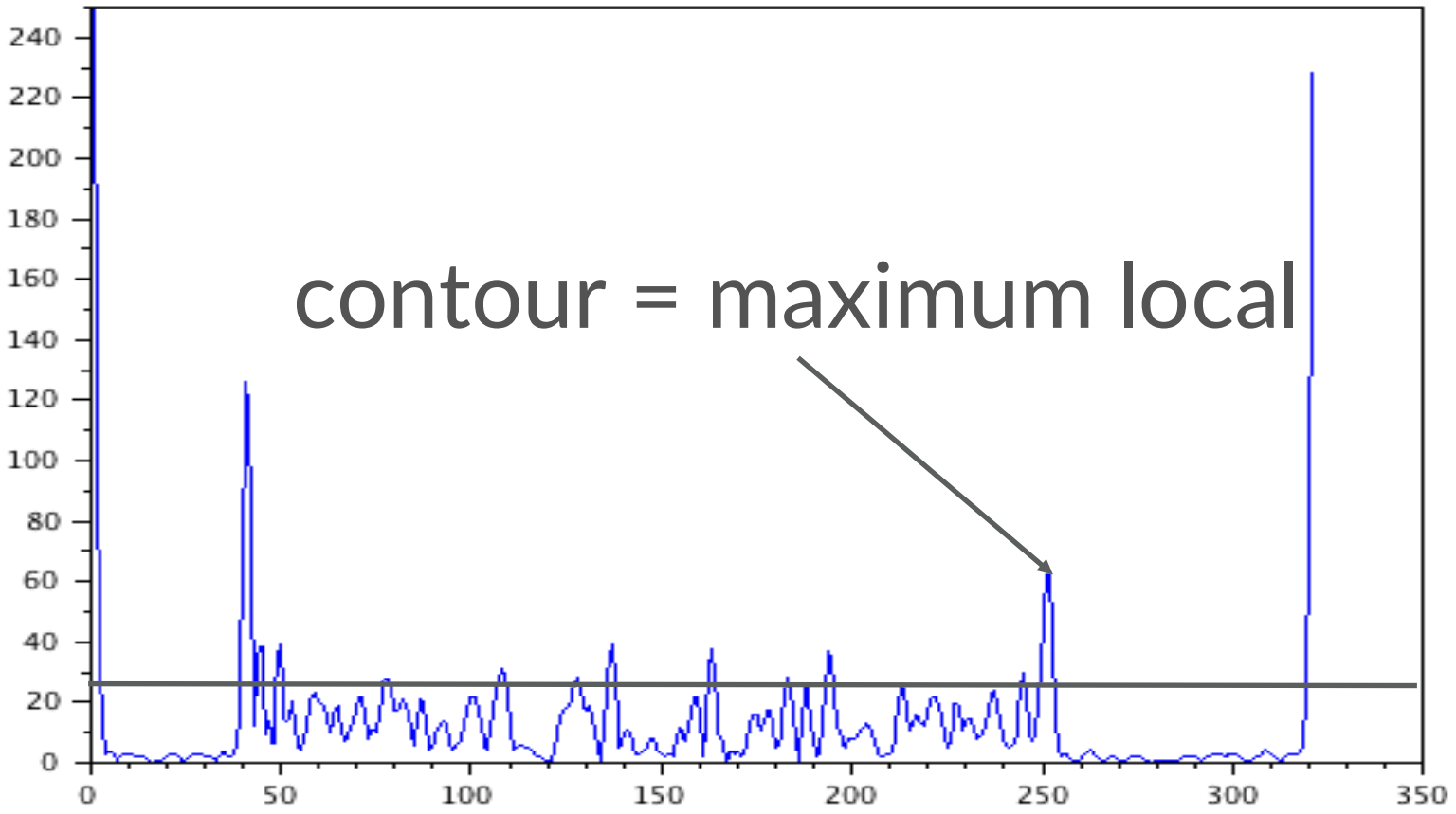


Mickael Nahon

Plan

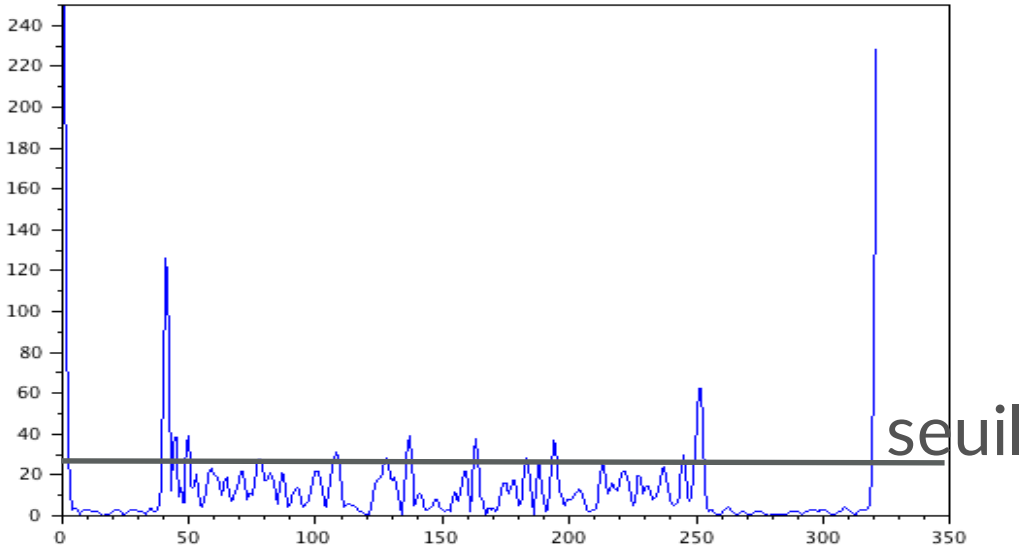
- Amélioration de la détection de contours
- Quelques éléments sur la compression d'images
- Quelques exercices type Examen

Amélioration de la détection de contours



Amélioration de la détection de contours

1) On identifie tous les pixels dont la norme du gradient
   est supérieure au seuil fixé



Amélioration de la détection de contours

1) On identifie tous les pixels dont la norme du gradient est supérieure au seuil fixé
   → (trop de) pixels autour desquels la luminosité varie

2) Quelle variations pour une direction quelconque d:

$$\frac{\partial I}{\partial d}(x,y) = < \nabla I(x,y) \, ; d >$$

- la variation est nulle si d est orthogonale au gradient
- la variation est maximale si elle est colinéaire

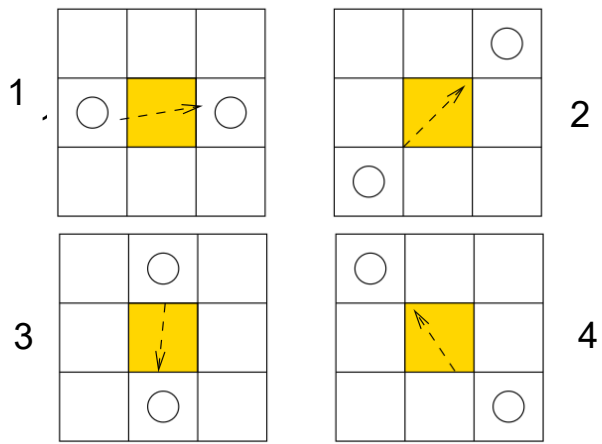
3) On observe la **direction du gradient** et on élimine les points qui ne sont pas des maximas locaux

En pratique

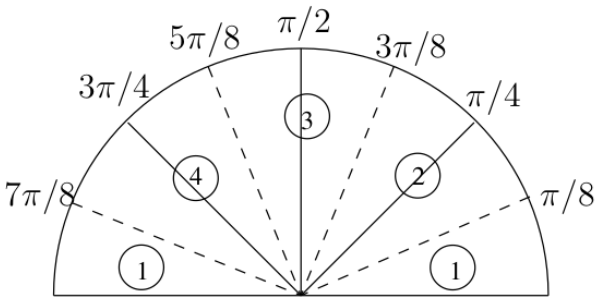
- A partir d'une image (filtrée), on calcule:
  - La matrice des dérivées en x: **imx**
  - La matrice des dérivées en y: **imy**
  - La matrice de la norme du gradient: **imn**
- La matrice des contours est définie comme:
  - **imc**(i,j) = 255 si **imn**(i,j) >= seuil et **imc**(i,j) = 0 sinon pour tout i,j

En pratique 2

- On raffine les contours en éliminant les ***non-maxima locaux***
  - Pour tout pixel (i,j) dans le contour i.e. avec **imc**(i,j) = 255
    - On calcule de l'angle ϕ du vecteur gradient [**imy**(i,j), **imy**(i,j)] avec l'axe x
    - $\phi = \tan^{-1}\left(\frac{imy[i,j]}{imx[i,j]}\right)$
    - On identifie les pixels adjacents ***dans la direction du gradient*** **=>** Deux pixels [i1,j1] et [i2,j2]
    - **Si imn**(i1,j1)>**imn**(i,j) **ou imn**(i2,j2)>**imn**(i,j). = > imc(i,j) = 0 (le pixel n'est pas un maximal local, on l'enlève du contour)
      Sinon imc(i,j) = 255 (le pixel reste dans le contour)

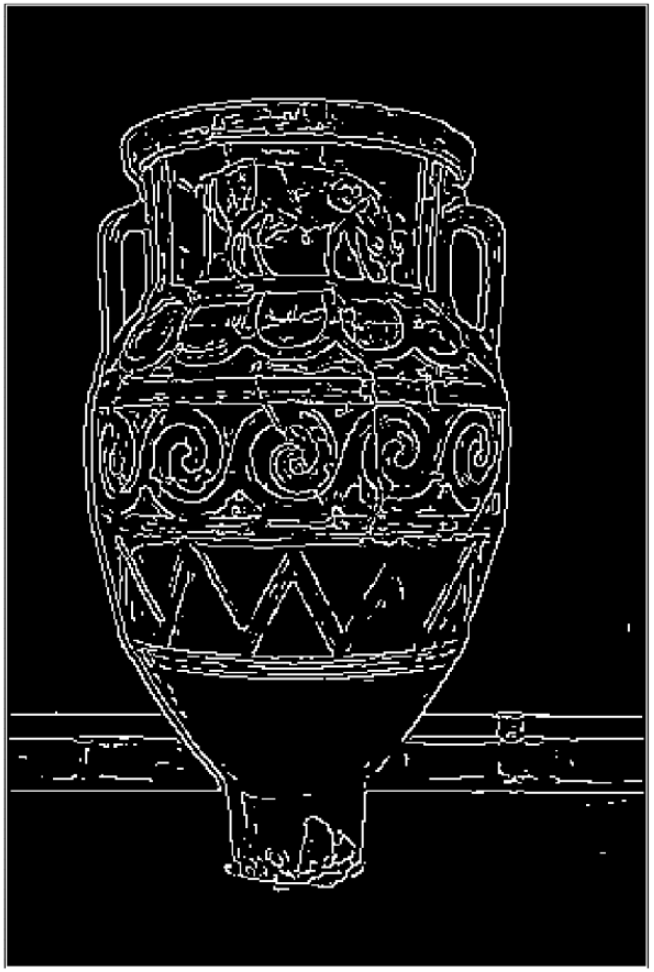


- Si $\phi \leq \pi/8$ ou $\phi \geq 7\pi/8$: cas 1.
- Si $\pi/8 < \phi \leq 3\pi/8$: cas 2.
- Si $3\pi/8 < \phi < 5\pi/8$: cas 3.
- Si $5\pi/8 \leq \phi < 7\pi/8$: cas 4.



Amélioration de la détection de contours





Après élimination des max. locaux, les contours sont
- plus fins
- mieux définis

→ TP

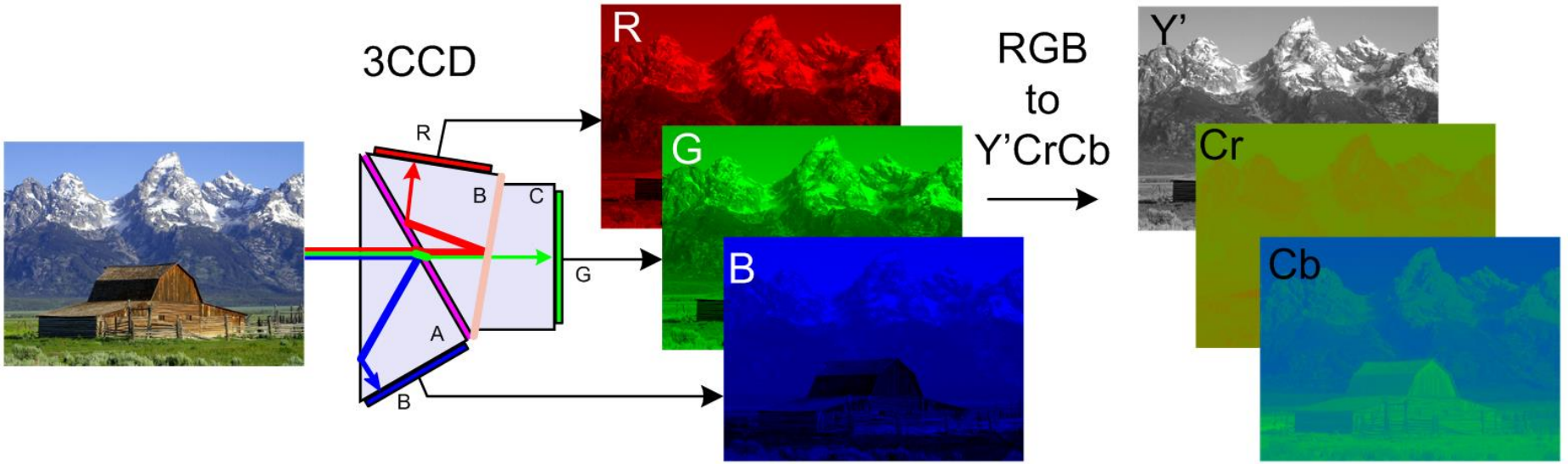
COMPRESSION D’IMAGES

Pourquoi compresser une image

- Les images brutes “raw” sont simplement stockées comme une liste de valeurs pixels (+la dimension de l’image et le format des pixels)
- Toute cette information n’est pas nécessaire
  - Pas forcément visualisable selon la définition de l’écran
  - Pas forcément visible par l’oeil humain
  - Pas forcément traitable par ordinateur

Images couleur

- RGB (red, green, blue) 8 bits/canal → 24 bits “true colors” utilisé par .bmp, .gif, .png
- Y’CbCr (Y’: gris + 2 “couleurs”) 8 bits/canal
  → meilleur visuel si dégradation, utilisé par .jpg
  Y'=0,299R+0,587G+0,114B
  Cb = − 0,1687R − 0,3313G + 0,5B + 128
  Cb=-0,1687R-0,3313G+0,5B+128



Principe de la compression

- On part d'une image brute

- Conception d'un codeur…
  - Algorithme/séquence d'opération qui va diminuer la taille mémoire de l'image (moins de nombres stockés)
  - Perte d'information autorisée

- … et d'un décodeur
  - L'image codée n'est pas représentable directement
  - On veut faire l'"inverse" du codage
  - On retrouve une version légèrement dégradée de l'image de départ

Redimensionnement

- Forme basique de compression
  - Utilisée en permanence pour afficher les images

- Réduction
  - On diminue par d*d en prenant 1 ligne/d et 1 colonne/d
  - Si on veut n pixels par ligne à partir d'une image à N pixels par ligne
    - *On prend les pixels* partie_entière_sup(i*N/n) *pour i=1,..,n*
    - *Pareil pour les colonnes*
  - Peut produire de l'aliasing → Filtre moyenneur appliqué au préalable

Redimensionnement



(1)



(2)



(3)

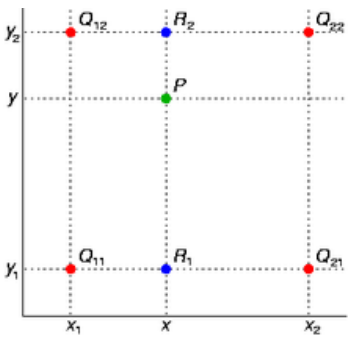
FIGURE 1.8 – (1) Une image numérique de taille 512×512, (2) La même imag sous-échantillonnée, en prenant 1 pixel sur 2 dans chaque dimension, (3) L même image filtrée, pour éliminer les hautes fréquences, puis sous-échantillonné en prenant 1 pixel sur 2 dans chaque dimension.

Redimensionnement

- Aggrandissement
  - Multiplier le nombre de pixels par a (en ligne et colonnes)
  - Trouver une *bonne* valeur de l'image en les coordonnées non-entières (i/a , j/a) pour i=1,..,a*M et j=1,..,a*N
- Interpolation bilinéaire
  - La valeur au point P=(x,y) est construite à partir de **4 pixels**
    x1<= x <= x2 (x1 et x2 entiers consécutifs)
    y1<= y <= y2
    → Les 4 pixels sont (x1,y1) (x1,y2) (x2,y1) (x2,y2)
  - Interpolation linéaire en 2D: trouver a,b,c,d t.q.

$$\begin{cases} f(x_1,y_1) = ax_1 + by_1 + cx_1y_1 + d \\ f(x_2,y_1) = ax_2 + by_1 + cx_2y_1 + d \\ f(x_1,y_2) = ax_1 + by_2 + cx_1y_2 + d \\ f(x_2,y_2) = ax_2 + by_2 + cx_2y_2 + d \end{cases}$$

$$f(x,y) = ax + by + cxy + d.$$

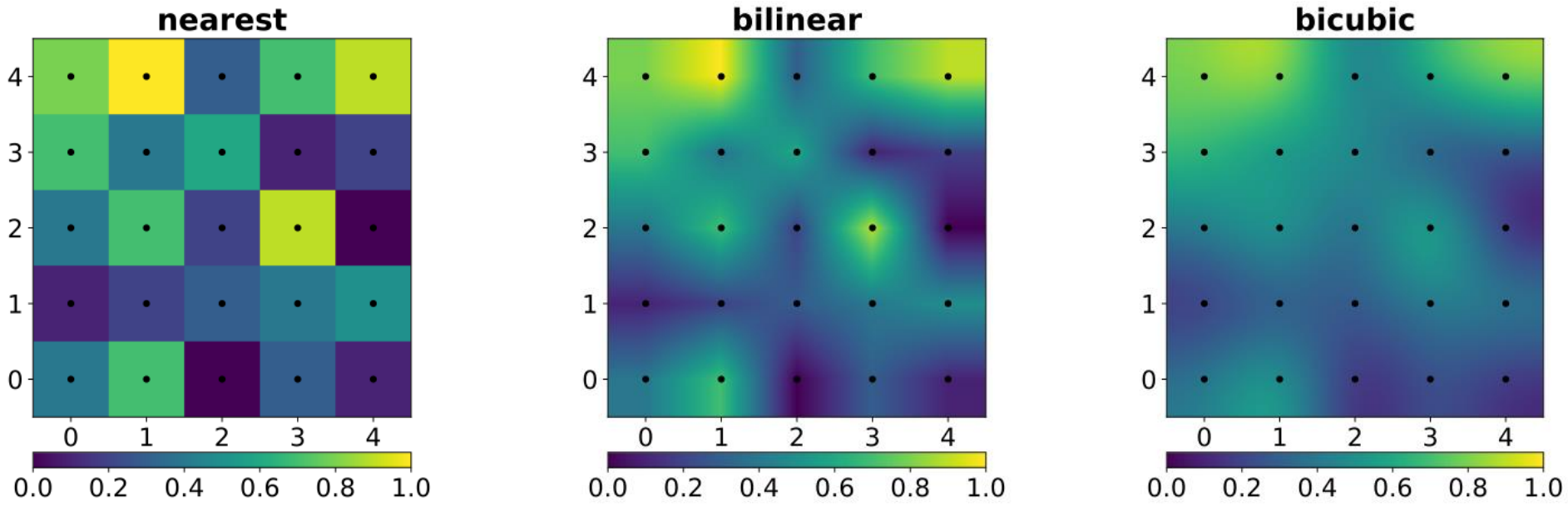


Redimensionnement

- Aggrandissement
  - Multiplier le nombre de pixels par a (en ligne et colonnes)
  - Trouver une *bonne* valeur de l'image en les coordonnées non-entières (i/a , j/a) pour i=1,..,a*M et j=1,..,a*N
- Interpolation bi**cubique**
  - Forme d'interpolation la plus utilisée
  - Garantie d'une certaine forme de régularité (Meilleur recollement)
  - Interpolation cubique en 2D: trouver 16 coefficients
    - *À partir de 4 pixels centraux: valeur, dérivée en x, dérivée en y, et dérivée "croisée"*
    - *4*4 = 16 valeurs et 16 coefficients à trouver*

$$p(x,y) = \sum\sum a_{ij}\, x^i y^j$$

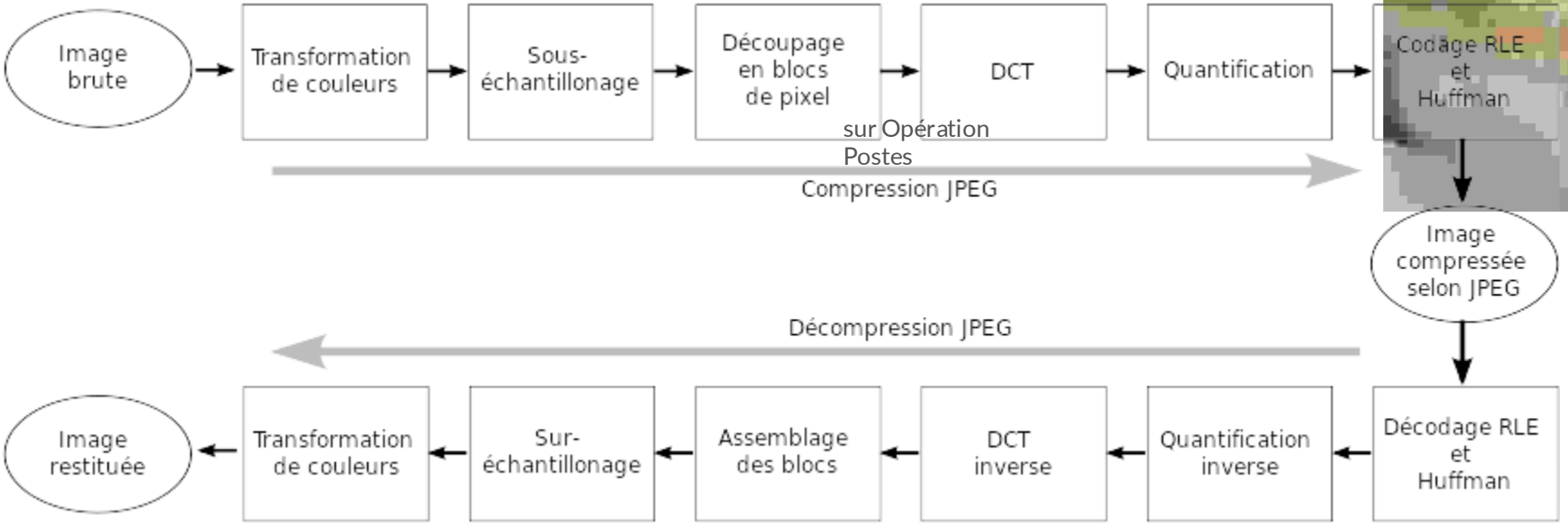
Redimensionnement



Compression JPEG

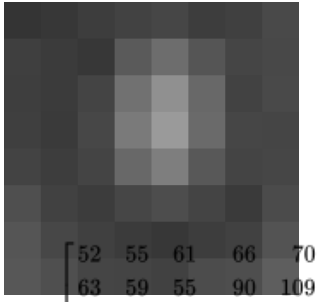
- Format populaire depuis mi 1990's





Compression JPEG 4: DCT

- “Discrete Cosine Transform”
- C’est l’étape clé de la compression
- Indépendamment sur chaque bloc: I ← Imagette – 128 (pour centrer)
- On passe d’une matrice I de taille 8x8
  à une matrice de fréquence G de taille 8x8 par



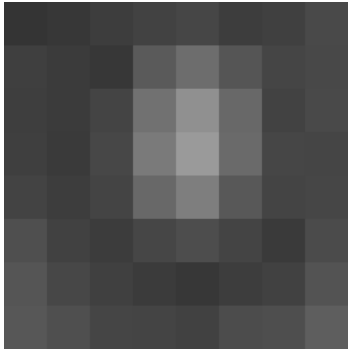
$$G(u,v) = \frac{1}{4}\sum \quad \sum \quad I(i,j)cos\left[\frac{(2i-1)u\pi}{16}\right]\cos\left(\frac{(2j-1)v\pi}{16}\right)$$

$$\begin{bmatrix} 52 & 55 & 61 & 66 & 70 & 61 & 64 & 73 \\ 63 & 59 & 55 & 90 & 109 & 85 & 69 & 72 \\ 62 & 59 & 68 & 113 & 144 & 104 & 66 & 73 \\ 63 & 58 & 71 & 122 & 154 & 106 & 70 & 69 \\ 67 & 61 & 68 & 104 & 126 & 88 & 68 & 70 \\ 79 & 65 & 60 & 70 & 77 & 68 & 58 & 75 \\ 85 & 71 & 64 & 59 & 55 & 61 & 65 & 83 \\ 87 & 79 & 69 & 68 & 65 & 76 & 78 & 94 \end{bmatrix}.$$

$$\begin{matrix} & \overset{x}{\longrightarrow} \\ \begin{bmatrix} -76 & -73 & -67 & -62 & -58 & -67 & -64 & -55 \\ -65 & -69 & -73 & -38 & -19 & -43 & -59 & -56 \\ -66 & -69 & -60 & -15 & 16 & -24 & -62 & -55 \\ -65 & -70 & -57 & -6 & 26 & -22 & -58 & -59 \\ -61 & -67 & -60 & -24 & -2 & -40 & -60 & -58 \\ -49 & -63 & -68 & -58 & -51 & -60 & -70 & -53 \\ -43 & -57 & -64 & -69 & -73 & -67 & -63 & -45 \\ -41 & -49 & -59 & -60 & -63 & -52 & -50 & -34 \end{bmatrix} & \downarrow y. \end{matrix} \quad G = \begin{matrix} & \overset{u}{\longrightarrow} \\ \begin{bmatrix} -415.38 & -30.19 & -61.20 & 27.24 & 56.12 & -20.10 & -2.39 & 0.46 \\ 4.47 & -21.86 & -60.76 & 10.25 & 13.15 & -7.09 & -8.54 & 4.88 \\ -46.83 & 7.37 & 77.13 & -24.56 & -28.91 & 9.93 & 5.42 & -5.65 \\ -48.53 & 12.07 & 34.10 & -14.76 & -10.24 & 6.30 & 1.83 & 1.95 \\ 12.12 & -6.55 & -13.20 & -3.95 & -1.87 & 1.75 & -2.79 & 3.14 \\ -7.73 & 2.91 & 2.38 & -5.94 & -2.38 & 0.94 & 4.30 & 1.85 \\ -1.03 & 0.18 & 0.42 & -2.42 & -0.88 & -3.02 & 4.12 & -0.66 \\ -0.17 & 0.14 & -1.07 & -4.19 & -1.17 & -0.10 & 0.50 & 1.68 \end{bmatrix} & \downarrow v. \end{matrix}$$

Compression JPEG 4: DCT

- Cela revient à projeter l'imagette sur la ***base DCT***

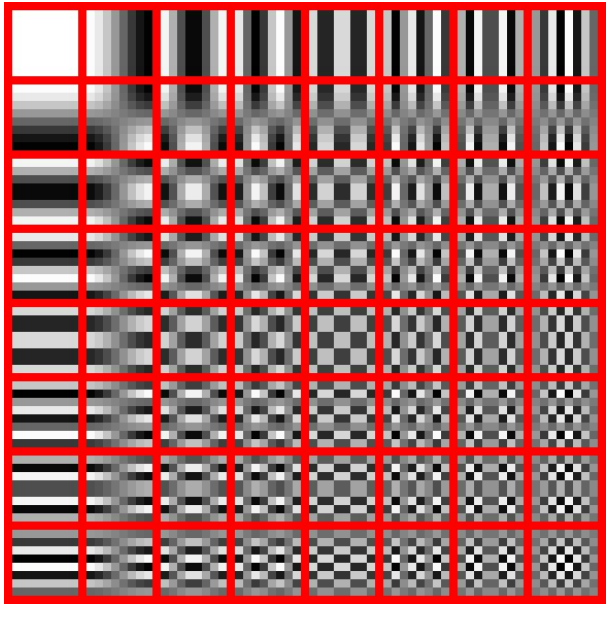
- imagette 

= coeffs

$$G = \begin{bmatrix} -415.38 & -30.19 & -61.20 & 27.24 & 56.12 & -20.10 & -2.39 & 0.46 \\ 4.47 & -21.86 & -60.76 & 10.25 & 13.15 & -7.09 & -8.54 & 4.88 \\ -46.83 & 7.37 & 77.13 & -24.56 & -28.91 & 9.93 & 5.42 & -5.65 \\ -48.53 & 12.07 & 34.10 & -14.76 & -10.24 & 6.30 & 1.83 & 1.95 \\ 12.12 & -6.55 & -13.20 & -3.95 & -1.87 & 1.75 & -2.79 & 3.14 \\ -7.73 & 2.91 & 2.38 & -5.94 & -2.38 & 0.94 & 4.30 & 1.85 \\ -1.03 & 0.18 & 0.42 & -2.42 & -0.88 & -3.02 & 4.12 & -0.66 \\ -0.17 & 0.14 & -1.07 & -4.19 & -1.17 & -0.10 & 0.50 & 1.68 \end{bmatrix}$$

($u \rightarrow$, $v \downarrow$)

Dans la base DCT



Compression JPEG 4: DCT

- C'est une opération réversible
  - A partir des coefficients et de la base, on retrouve l'image
  - La base DCT est une base de l'espace des matrices 8x8
- Formule de l'inverse

$$I(i,j) = \frac{1}{4}\sum\sum G\,(u,v)cos\left[\frac{2\mathrm{i}u\pi}{16}\right]cos\left[\frac{2\mathrm{j}v\pi}{16}\right]$$

- La DCT est un changement de réprésentation de l'image et pas une compression
  - Matrice 8x8 entiers 0..255 ↔ Matrice 8x8 réels

Compression JPEG 5: Quantification

- Pour gagner en taille, on quantifie ***les coefficients DCT (G)***
  - Meilleure qualité visuelle que sur l'imagette
  - Les tendances générales sont respectées
- On utilise une matrice de quantification fixe **Q** (standard JPEG 50% ici)
  - La matrice quantifiée ***B*** est définie comme B(i,j) = arrondi(G(i,j)/Q(ij))
  - Perte d'information: les hautes fréquences sont supprimées
  - La matrice B a des coeff. entiers et beaucoup de zéros

$$G = \begin{bmatrix} -415.38 & -30.19 & -61.20 & 27.24 & 56.12 & -20.10 & -2.39 & 0.46 \\ 4.47 & -21.86 & -60.76 & 10.25 & 13.15 & -7.09 & -8.54 & 4.88 \\ -46.83 & 7.37 & 77.13 & -24.56 & -28.91 & 9.93 & 5.42 & -5.65 \\ -48.53 & 12.07 & 34.10 & -14.76 & -10.24 & 6.30 & 1.83 & 1.95 \\ 12.12 & -6.55 & -13.20 & -3.95 & -1.87 & 1.75 & -2.79 & 3.14 \\ -7.73 & 2.91 & 2.38 & -5.94 & -2.38 & 0.94 & 4.30 & 1.85 \\ -1.03 & 0.18 & 0.42 & -2.42 & -0.88 & -3.02 & 4.12 & -0.66 \\ -0.17 & 0.14 & -1.07 & -4.19 & -1.17 & -0.10 & 0.50 & 1.68 \end{bmatrix} \begin{matrix} \\ \\ \\ \\ \\ \\ \\ \downarrow v. \end{matrix} \quad \overset{u}{\longrightarrow}$$

$$Q = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}.$$

$$B = \begin{bmatrix} -26 & -3 & -6 & 2 & 2 & -1 & 0 & 0 \\ 0 & -2 & -4 & 1 & 1 & 0 & 0 & 0 \\ -3 & 1 & 5 & -1 & -1 & 0 & 0 & 0 \\ -3 & 1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Compression JPEG

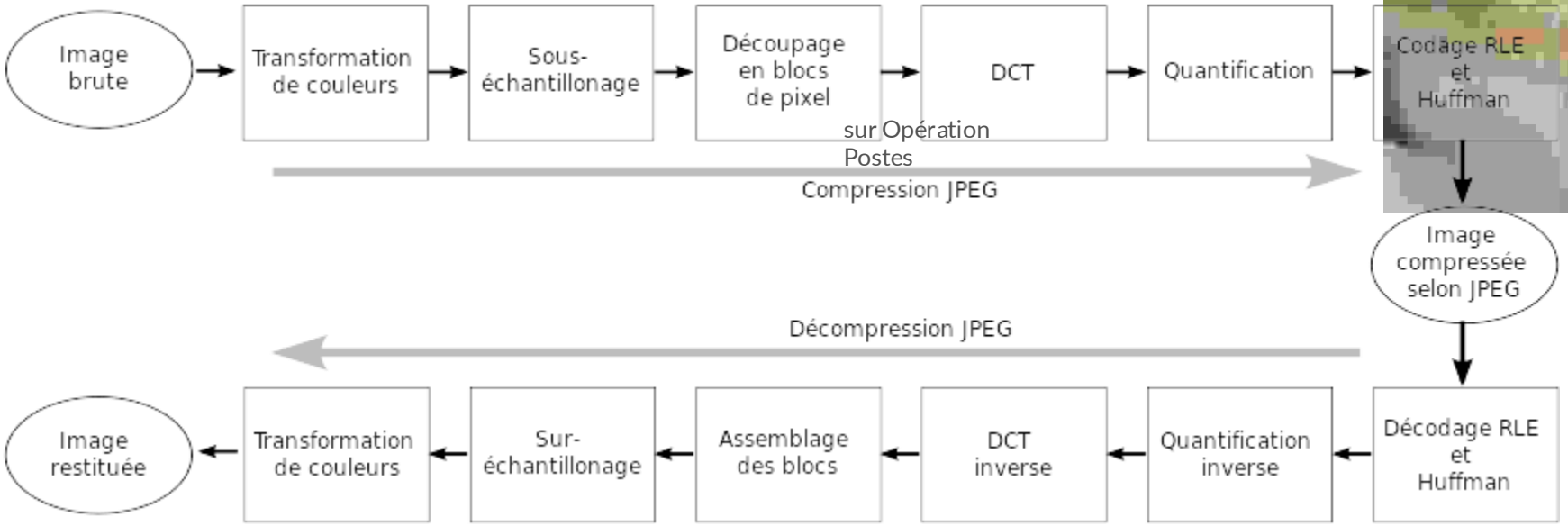
- Image (313 × 234 pixels) x 3 couleurs → 219,726 bytes

Image	Quality	Size (bytes)	Compression ratio	Comment
	Highest quality (Q = 100)	81,447	2.7:1	Extremely minor artifacts
	High quality (Q = 50)	14,679	15:1	Initial signs of subimage artifacts
	Medium quality (Q = 25)	9,407	23:1	Stronger artifacts; loss of high frequency information
	Low quality (Q = 10)	4,787	46:1	Severe high frequency loss leads to obvious artifacts on subimage boundaries ("macroblocking")
	Lowest quality (Q = 1)	1,523	144:1	Extreme loss of color and detail; the leaves are nearly unrecognizable.

Compression JPEG

- Format populaire depuis mi 1990's





EXERCICES

Ex. 1

1. Le filtre $\mathrm{H} = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$ est-il un filtre de moyennage local, de gradient en x (horizontal) ou de gradient en y (vertical). Justifiez.

2. Appliquer le filtre H à l'image ci-dessous:

0.	0.	0.	0.	0.	0.
0.	0.	0.	100.	100.	100.
0.	0.	0.	100.	100.	100.
0.	0.	0.	100.	100.	100.
0.	0.	0.	0.	0.	0.

Ex. 2

On considère une image de taille 5x5 avec 5 niveaux de gris, de 0 (noir) à 4 (blanc)



```
im  =
    2.    0.    3.    3.    0.
    2.    3.    0.    0.    2.
    4.    1.    2.    3.    1.
    0.    2.    1.    3.    1.
    2.    4.    1.    1.    2.
```

1. Tracer l'histogramme de cette image. ATTENTION : valeurs des pixels de 0 à 4.
2. Tracer l'histogramme cumulé.
3. Appliquer la transformation T(p) = 2*p -2
4. Appliquer l'égalisation d'histogramme

Ex. 3

On considère le filtre $H = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$, dont l'effet sur une image est d'augmenter sa netteté apparente:



Image initiale



Image transformée

Le but de l'exercice est de comprendre son fonctionnement.

On considère le filtre $\texttt{H} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$, dont l'effet sur une image est d'augmenter sa netteté apparente :



Image initiale



Image transformée

Le but de cet exercice est de comprendre son fonctionnement à partir d'un exemple simple.

Soit `I1` une image de taille `M x N` et soit `I2 = conv(I1, H)` l'image produite par convolution de `I1` avec `H` ; on la définit en `scilab` de la manière suivante : pour tout `u` allant de `2` à `M-1`, et pour tout `v` allant de `2` à `N-1`,

```
I2(u,v) = sum(I1(u-1:u+1, v-1:v+1).*H);
```

et `I2(u,v) = I1(u,v)` pour les autres valeurs de `(u,v)`.

Ex. 3

1. Expliquer pourquoi le filtre H ne modifie pas la luminosité globale de l'image.
2. Afin d'étudier le comportement de ce filtre sur un contour de l'image, on s'intéresse à une image élémentaire I1 qui présente une variation d'intensité sur quelques pixels dans la direction horizontale. Au sein d'une image plus complexe, cette image élémentaire pourrait être le bord d'un objet.

```
I1  =

    50.    50.    50.    50.   100.   150.   200.   200.   200.   200.
    50.    50.    50.    50.   100.   150.   200.   200.   200.   200.
    50.    50.    50.    50.   100.   150.   200.   200.   200.   200.
    50.    50.    50.    50.   100.   150.   200.   200.   200.   200.
    50.    50.    50.    50.   100.   150.   200.   200.   200.   200.
```

   Calculer l'image I2 = convolve2d(I1, H)

3. En s'appuyant sur cet exemple, justifier pourquoi on qualifie ce filtre de réhausseur de contour.
4. Quel serait le comportement de ce filtre vis-à-vis du bruit de l'image ?

1. Expliquez pourquoi le filtre `H` ne modifie pas la luminosité globale de l'image.

2. Afin d'étudier le comportement de ce filtre sur un contour de l'image, on s'intéresse à une image élémentaire `I1` qui présente une variation d'intensité sur quelques pixels dans la direction horizontale ; au sein d'une image plus complexe, cette image élémentaire pourrait être le bord d'un objet.

```
I1  =
   50.   50.   50.   50.   100.   150.   200.   200.   200.   200.
   50.   50.   50.   50.   100.   150.   200.   200.   200.   200.
   50.   50.   50.   50.   100.   150.   200.   200.   200.   200.
   50.   50.   50.   50.   100.   150.   200.   200.   200.   200.
   50.   50.   50.   50.   100.   150.   200.   200.   200.   200.
```

   Calculez l'image `I2 = conv(I1, H)`.

3. En vous appuyant sur cet exemple, justifiez pourquoi on qualifie ce filtre de *réhausseur de contour*.

4. Quel serait le comportement de ce filtre vis-à-vis du bruit de l'image ?

Ex. 4

1) Nous considérons 4 filtres 2D appliquées à l'image du papillon utilisée maintes fois en TP. Quel filtre correspond à quel résultat? Votre réponse devra être justifiée.

Filtres :

$$\textbf{1:}\ \begin{matrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{matrix} \qquad \textbf{2:}\ \frac{1}{9}\begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix} \qquad \textbf{3:}\ \begin{matrix} 0 & 0.5 & 0 \\ 0.5 & 1 & 0.5 \\ 0 & 0.5 & 0 \end{matrix} \qquad \textbf{4:}\ \begin{matrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{matrix}$$

A:



B:



C:



D:

